

DuitLater · Technical Pitch Deck · TNG FINHACK 2026

Multi-cloud architecture · HA failover · 3-layer AI · ~18 slides

Team R2-D2

DuitLater — Technical Deck

Multi-cloud · HA failover · BM-native AI

Companion to the 4-min business pitch. Deeper architectural detail for technical evaluation, sponsor architects, and post-pitch Q&A.

Stack: Hono · Prisma · PostgreSQL 17 · Next.js 15 · Tailwind v4 · Better Auth · Docker Compose · Caddy · GitHub Actions **Multi-cloud:** AWS (compute · storage) + Alibaba Cloud (AI inference · DR mirror) **Architecture style:** Active-passive HA · streaming replication · serverless AI · cross-cloud backup

What We're Solving

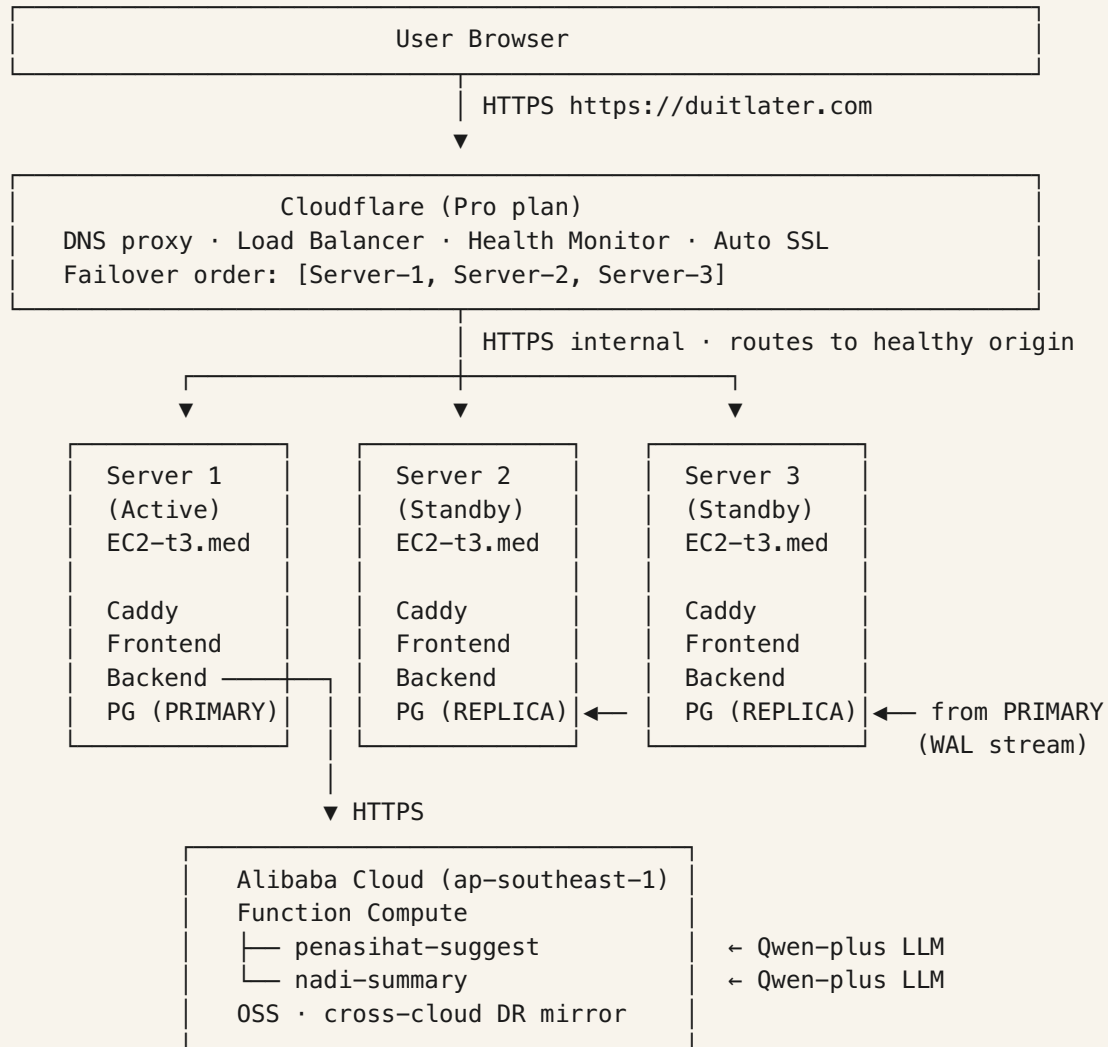
A B40 wallet has a small PayLater limit. A B40 household needs bigger items.

The constraint TNG has today: - Every TNG eWallet user has individual PayLater allowance - Allowance based on individual financial track record - For B40 households, that's typically RM 100–RM 500 - Enough for groceries, insufficient for sewing machine, generator, agricultural tools, bulk school supplies

The mechanic that's missing: - No way for 2-8 users to combine their allowances - No way for community trust score to influence individual capacity - No way for institutional partners (NADI, MyKasih) to facilitate pool formation

DuitLater is that wiring.

System Architecture • 30,000-foot View



Tech Stack · By Layer

Layer	Choice	Why
Repo	pnpm monorepo · <code>packages/{backend,frontend,db}</code>	Shared types · single lockfile · workspace deps
Runtime	Node 22 (engines: ≥ 18) · pnpm ≥ 9	Modern · stable · monorepo-native
Backend HTTP	Hono 4 + <code>@hono/node-server</code>	Faster than Express · TypeScript-native · zod-validator integration
ORM	Prisma + <code>@prisma/adapters-pg</code>	Type-safe schema → migrations → client · separate <code>db</code> workspace package
Database	PostgreSQL 17 alpine	Streaming replication built-in · WAL discipline · open source
Auth	Better Auth (planned) · argon2 hashing	OWASP-recommended hashing · session-based
Validation	zod 4 · Hono zod-validator	Single source of truth · request + response schema
Frontend	Next.js 15 + React 19 · App Router	Server components · streaming · production-grade
Styling	Tailwind v4 + tw-animate-css	CSS-first config (<code>@theme</code>) · faster than v3
State	TanStack Query 5 + Jotai 2	Server state separated from client state · atom-based
Forms	react-hook-form + <code>@hookform/resolvers</code> + zod	Type-safe · controlled · performant
PWA	Serwist 9 + <code>@serwist/next</code>	Service worker · offline-capable for B40 mobile
Reverse proxy	Caddy 2 alpine	Auto Let's Encrypt · path-routing · zero-config TLS
Container	Docker + Docker Compose plugin	Standard · reproducible · multi-stack on single host
CI/CD	GitHub Actions → GHCR	<code>backend-release.yml</code> · <code>ci-backend.yml</code> · <code>ci-frontend.yml</code>

Why Multi-Cloud (And Not Just AWS)

The deliberate split:

Cloud	Role	Reason
AWS <i>(Gold sponsor)</i>	Main compute · Postgres ledger · S3 storage	Mature single-region deploy · familiar ops surface · existing infra automation via GitHub Actions → GHCR pipeline
Alibaba Cloud <i>(Platinum sponsor)</i>	AI workloads (Function Compute · Qwen LLM · OSS)	(1) Qwen is BM-native — handles Bahasa Melayu reasoning more accurately than English-trained models. (2) Serverless cost-optimised for small structured-output workloads. (3) Data sovereignty — B40 user financial context stays in regional sovereign cloud. (4) Sponsor alignment.

Failover preserved: - Backend service router (`services/penasihat.ts` , `services/nadi-summary.ts`) calls Alibaba Function Compute primary - Falls back to **Anthropic Claude** on 5xx or timeout (>6s) - Both providers receive identical structured-output schema - Provider used returned in response for observability

Multi-cloud is real, not theatrical — actual code routing, deployable Function Compute handler in repo, env vars wired across all 3 EC2 servers.

HA Failover · 3-Server Active-Passive

Topology

- **3 EC2** in AWS ap-southeast-1 · each runs full DuitLater stack
- **Cloudflare LB** with failover-order steering policy
- Priority: `[Server-1, Server-2, Server-3]`
- Health check: HTTPS GET `/api/v1/health` · 30s interval · 2 retries → 90s detection time

Failover scenarios

Scenario	Detection	Action	Recovery
All healthy	—	All traffic → Server 1	—
Server 1 down	~90s	Cloudflare routes to Server 2	Auto on Server 1 healthy again
Server 1 + 2 down	~90s	Cloudflare routes to Server 3	Auto on first healthy
All 3 down	~90s	Cloudflare 521 / fallback page	Manual intervention required
Server 1 returns	~60s healthy	Cloudflare returns to Server 1	Auto

Database write failover

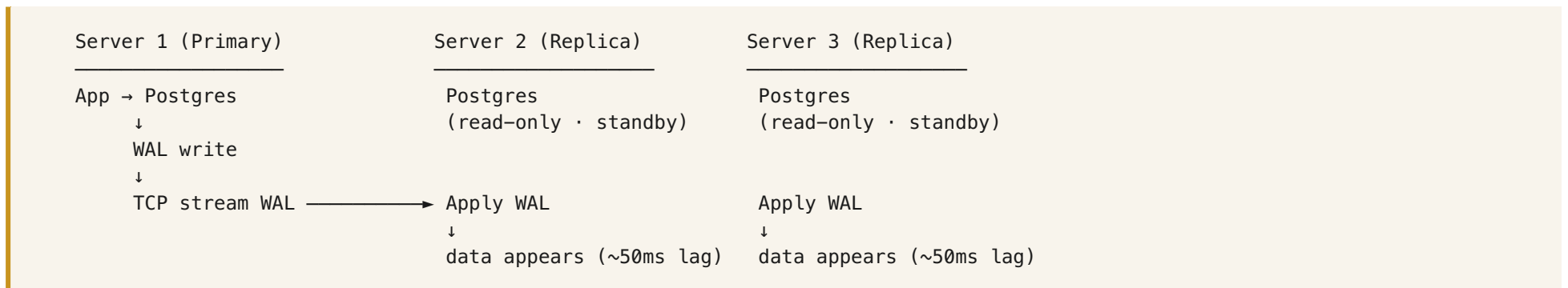
- Auto web routing ≠ auto DB write capability
- Replicas are **read-only** until manually promoted
- DB promotion playbook: `pg_ctl promote` · ~10 second command via SSH
- Post-promotion: re-base original primary as replica from new primary

- Total downtime for writes: ~2 minutes (90s detection + 30s promote + 30s backend restart)

PostgreSQL

Streaming Replication

How data sync works



Configuration choices

- **Async streaming** (default) — primary doesn't wait for replica ACK · faster writes · ~50-100ms lag · tiny risk of last-100ms write loss on primary crash
- **Sync streaming** (optional) — primary waits for replica ACK · slower writes · zero data loss
- For hackathon: **async** acceptable (write throughput > guarantee)

Setup commands (compressed)

```
# On primary (Server 1)
ALTER USER duitlater REPLICATION;
CREATE USER replicator WITH REPLICATION ENCRYPTED PASSWORD '<pwd>';
# postgresql.conf: wal_level=replica, max_wal_senders=5, wal_keep_size=1GB
# pg_hba.conf: allow host replication replicator <server-2-ip>/32 md5

# On replicas (Server 2 + 3)
pg_basebackup -h <server-1-ip> -U replicator -D /var/lib/postgresql/data -X stream -R
# -R flag auto-creates standby.signal + primary_conninfo

# Verify on primary
SELECT * FROM pg_stat_replication;
```

AI Layer 3 · Penasihat Suggester (Multi- Cloud)

Request flow


```
Pool member clicks "Cadangkan barang"
↓
POST /api/v1/penasihat/suggest { poolId }
↓
Backend (Server 1 · Hono):
  1. Fetch pool context: combinedCap, statedNeed, kampungName, monthOfYear
  2. Query mykasih_catalogue for items <= combinedCap
  3. Route AI call:
      └─ if ALIBABA_FUNCTION_COMPUTE_URL set:
          → POST to Alibaba FC (timeout 6s)
          → Function calls Qwen-plus via DashScope API
          → Returns structured 5 suggestions
          └─ on 200: return to user
            └─ on 5xx/timeout: fallback to Claude
      └─ else: call Anthropic Claude directly
  4. Cache to pool_suggestions table for 30 min
  5. Return suggestions + provider field for observability
```

Why Qwen primary, not Claude

- **Bahasa Melayu reasoning quality** — Qwen handles BM register more naturally
- **Cost-optimised** for small structured-output workloads (~RM 0.001/request vs RM 0.005)
- **Regional sovereignty** — Alibaba Cloud KL/SG region keeps B40 data in regional sovereign infra
- **Sponsor-aligned** — Alibaba Cloud is FINHACK 2026 Platinum sponsor

Failover semantics

```
// services/penasihat.ts
if (env.ALIBABA_FUNCTION_COMPUTE_URL) {
  try {
    const items = await callAlibabaFunctionCompute(ctx, candidates);
    return { items, provider: "alibaba-qwen" };
  } catch (err) {
    logger.warn({ err }, "Alibaba FC failed; falling back to Anthropic Claude");
  }
}
const items = await callClaude(ctx, candidates);
return { items, provider: "anthropic-claude" };
```

AI Layer 3 • NADI Weekly Summary (Anomaly Detection)

Surface

NADI staff opens [/nadi/dashboard](#) . “Ringkasan Minggu” card surfaces an AI-generated weekly digest.

What AI does

Receives weekly context: - Pools formed this week (count + breakdown by category) - Top-requested items (top 5) - Kampung trust score Δ vs last week - Late-payment events (member counts + days late)

Returns BM-first summary: - **Headline** (1 sentence) - **3-5 observations** (bullet points) - **0-3 anomalies** flagged (clusters of 3+ late payments → kampung-distress signal) - **BM-first action suggestion**

Why this earns the AI criterion deeper

This is **longitudinal pattern surfacing**, not single-shot inference. AI looks across weekly state to identify what humans wouldn't easily spot at scale:

- 188 NADI centres × weekly cycles = 9,776 weekly contexts annually
- Cost-optimised serverless inference scales transparently

- Same multi-cloud routing pattern (Alibaba primary, Claude fallback)

AI Methodology • Three Layers

DuitLater integrates AI across three project lifecycle layers

Layer 1 — Pre-product

~2,400 lines of planning artifacts authored AI-collaboratively before product code began:

- PRD.md (617 lines)
- ARCHITECTURE.md (mermaid diagrams · ER · multi-cloud topology)

- DEVELOPMENT-PLAN.md (7-phase build · testable outcomes · cut-line strategy)
- BRAND.md · TEAM.md · WORLD.md · pitch deck · narration · multi-cloud setup guide (1,400 lines)

Every doc in this repo is evidence.

Layer 2 — Process

`maji-core/` team coordinator ships with the repo:

- 6 slash commands: `/maji-onboard` `/maji-whoami` `/maji-phase` `/maji-gate` `/maji-pair` `/maji-handoff`
- Schema-locked persistent memory (per-member JSON, committed to git)
- Phase gates (Kairu's ladder — refuses advancement without testable outcome verification)
- Akal coding discipline (THINK · SIMPLE · SURGICAL · VERIFY)
- Jimat communication register (3 compression modes · default penuh)

Layer 3 — In-product

- **Penasihat** catalogue suggester (multi-cloud · BM-native)
- **NADI weekly summary** with anomaly detection (multi-cloud · longitudinal)

“Effective and meaningful integration of AI” — DuitLater hits this at three independent layers. Each layer demonstrably runs in repo. No layer is theatrical.

Data Model • Pool Lifecycle

Postgres schema (Prisma)


```

model User {
  id          String    @id @default(cuid())
  email       String    @unique
  name        String
  kampungId   String?
  individualPaylaterCents Int
  role        UserRole @default(MEMBER)
  ...
}

model Kampung {
  id          String    @id @default(cuid())
  name        String
  nadiCentreName String
  district    String
  trustScore  Decimal @default(60)
  ...
}

model Pool {
  id          String    @id @default(cuid())
  kampungId   String
  initiatorUserId String
  name        String
  statedNeed  String
  category    Category
  combinedCapCents Int
  state       PoolState @default(DRAFT)
  ...
}

model MykasihProduct {
  id          String    @id @default(cuid())
  name        String
  nameMs      String?
  category    MykasihCategory
  priceRm     Decimal
  subsidyRm   Decimal

```

```
...  
}
```

Pool state machine

```
draft → locked → suggesting → voting → approved → active → completed  
                                     ↓  
                                   dissolved
```

Forward-only transitions. Each transition logged with timestamp + initiator.

Invariants

- All money columns: integer cents (never float)
- `paylater_obligations` rows append-only after creation
- `repayments` append-only · corrections via compensating rows
- Pool `combined_cap_cents` set at lock time · never recalculated

- `kampung_trust_scores` recalculated on every repayment or pool completion event

Cross-Cloud Backup Strategy

Hourly Postgres → AWS S3

```
# /opt/duitlater/backup-pg.sh – runs on Server 1 every hour
docker exec duitlater-prod-postgres \
  pg_dump -U duitlater -d duitlater --format=custom --compress=9 \
  > /opt/duitlater/backups/duitlater-$(date -u +%Y%m%dT%H%M%S).sql.gz

aws s3 cp ... s3://duitlater-postgres-backups/$(date -u +%Y/%m/%d)/...
--storage-class STANDARD_IA
```

S3 lifecycle: delete after 30 days. Standard-IA storage class for cost.

Daily S3 → Alibaba OSS mirror

```
# /opt/duitlater/mirror-to-oss.py - runs daily at 02:00 UTC
import boto3, oss2

s3 = boto3.client("s3", region_name="ap-southeast-1")
oss = oss2.Bucket(auth, "https://oss-ap-southeast-1.aliyuncs.com",
                  "duitlater-backup-mirror")

for obj in s3.list_objects_v2(Bucket=S3_BUCKET, Prefix=today)["Contents"]:
    if not oss.object_exists(obj["Key"]):
        body = s3.get_object(Bucket=S3_BUCKET, Key=obj["Key"])["Body"].read()
        oss.put_object(obj["Key"], body)
```

Why cross-cloud DR matters

- AWS region down → Alibaba OSS still has yesterday's backup
- 30-day retention on both clouds

- Restore drill: test once a month via `pg_restore` to throwaway container

Security Posture · By Layer

Authentication & sessions

- argon2 password hashing (Better Auth default · OWASP-recommended)
- HttpOnly Secure SameSite=Lax cookies — no JS access to tokens
- Session rotation on auth event
- Role-based access control (`member` vs `nadi_staff`)

Webhook integrity

- HMAC verification on TNG webhook callbacks before any state mutation
- Idempotency keys prevent double-processing
- Unverified webhooks dropped without logging payload (no leakage)

Money math

- All amounts in **integer cents** — no floating-point drift
- `paylater_obligations` and `repayments` append-only
- Corrections via compensating rows referencing originals

Data sovereignty

- B40 user financial context flows to **Alibaba Cloud** (regional sovereign) via Function Compute, not US-based AI providers as primary
- PII minimised in AI prompts — only first name + numeric pool context + stated need
- Anthropic Claude as failover only · explicit downgrade path with logging

Network security

- Postgres never publicly exposed — Docker internal network only
- Frontend + backend bound to Docker network — only Caddy speaks public
- HTTPS-only via Caddy automatic SSL

- Rate limiting via `hono-rate-limiter` on auth + AI endpoints

Production Scale Path

MVP → pilot → national

T0 (Hackathon · MVP)

EC2 t3.medium × 3
Postgres replication (3-way)
Direct Alibaba FC calls
S3 hourly backups

T1 (Pilot · 1–5 NADI · 100s pools)

EC2 t3.large + CloudFront CDN
Postgres + read replica
Redis queue (BullMQ for async TNG)
Alibaba FC auto-scales transparently
S3 + cross-cloud OSS daily

T2 (National · 188 NADI · 10k+ pools)

ALB + Auto Scaling Group + Aurora
Aurora multi-AZ + auto-scaling readers
SQS + Lambda
Same · auto-scales transparently
Same · multi-region replication

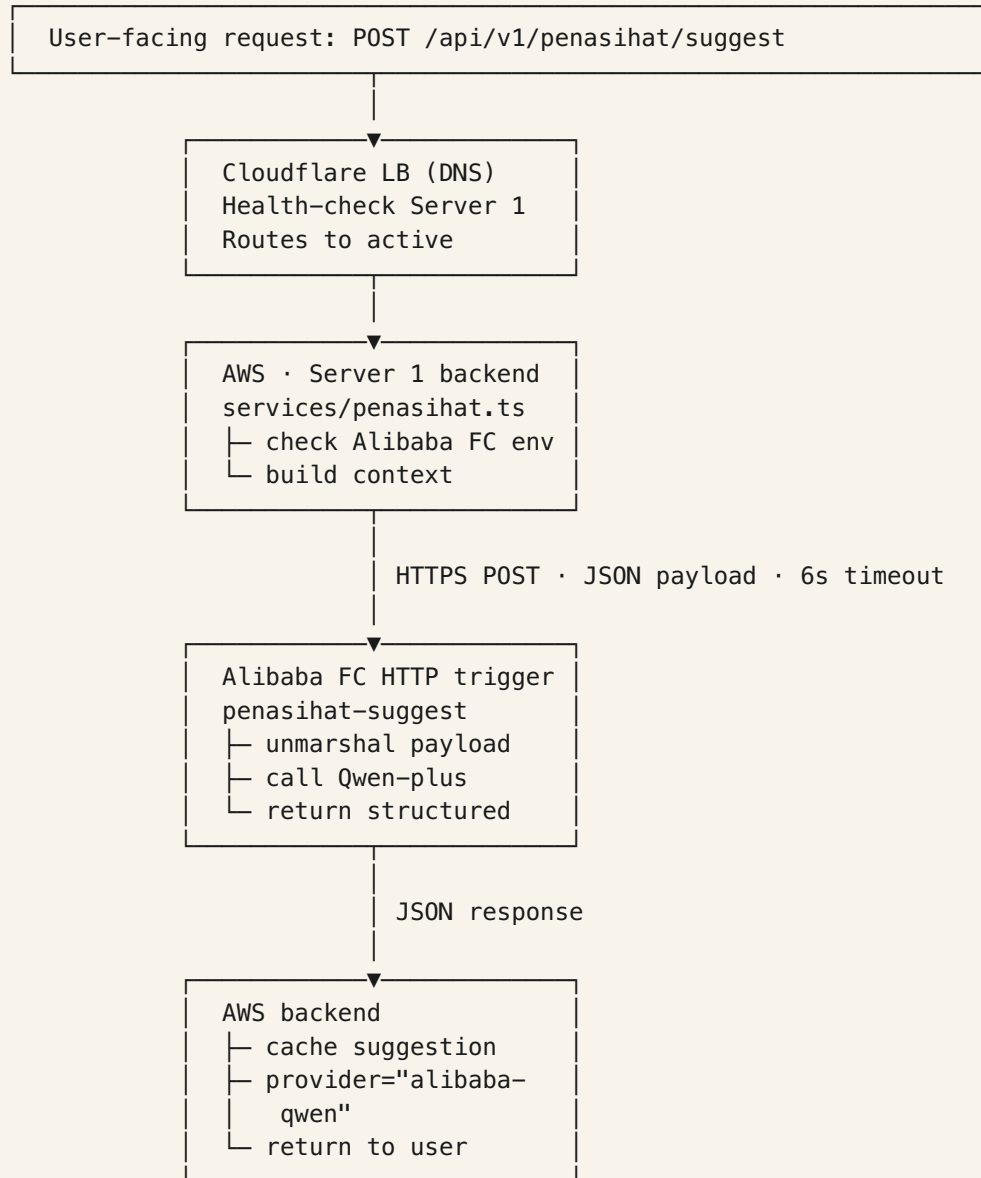
Bottleneck → fix matrix

Bottleneck	MVP behaviour	Pilot fix	National fix
Compute	Single EC2 × 3 (HA)	Vertical scale → t3.large	ASG + ALB
DB writes	Single primary	Read replica for queries	Aurora multi-AZ auto-scaling
AI inference	Synchronous Alibaba FC	Cache common suggestions	FC auto-scales
TNG webhooks	Inline processing	Redis queue	SQS + Lambda
Static assets	Caddy local	CloudFront CDN	Same · multi-region
MyKasih catalogue	Seeded (30 items)	Sync job nightly	Real-time webhook

Key invariant: Postgres remains canonical for the **append-only ledger**. All scale moves happen *around* the ledger, not replacing it. Audit-ability + regulatory readiness preserved.

Multi-Cloud Orchestration • Detailed Flow

How AWS + Alibaba coordinate



Coordination guarantees

- **Idempotency** — Same poolId returns cached suggestion within 30 min (no Alibaba re-charge)
- **Failover transparency** — User never knows whether Qwen or Claude served the request
- **Observability** — `provider` field in response logs which path was used
- **Rate limit safety** — `hono-rate-limiter` caps per-user requests · prevents Alibaba quota burn

Demo Strategy · Stage Walkthrough

What judges see in 60 seconds (slide 5 of business deck)

Step 1 (8s): Daftar sebagai Mak Cik Aminah. Dashboard tunjuk PayLater individual: RM 300.

Step 2 (10s): Cipta pool “Felda Gedangsa Mac” · need: “Mesin jahit untuk side income” · target RM 1,800.

Step 3 (12s): Generate invite code, share. Two neighbours join via incognito browsers. Combined cap: RM 1,800 → RM 2,000.

Step 4 (15s): Click “Lock pool”. Click “Cadangkan barang”. 5 BM suggestions render — *via Alibaba Cloud Function Compute · provider field shows “alibaba-qwen”*. Pick mesin jahit.

Step 5 (10s): Pool vote: 3-of-4 yes. Simulated TNG approval per member’s proportional share. NADI staff (separate login) opens portal, clicks “Sahkan dah hantar”.

Step 6 (5s): Cycle 1 begins. Member clicks “Bayar”. Ledger row turns green. Kampung trust score: 85 → 86.

Hidden orchestration (judges can verify post-pitch)

- Penasihat call traverses AWS → Alibaba Cloud (multi-cloud · live)
- Postgres replication lag visible via `pg_stat_replication` (sub-100ms)
- Cloudflare LB analytics show 3 healthy origins

- All 3 EC2 IPs hidden behind single Cloudflare proxy

Risks & Honest Limitations

Risk	Likelihood	Impact	Mitigation
TNG sandbox not provisioned by Day 1	Medium	High	Backend abstracts payment provider · simulated client returns success
Claude API rate-limit during demo	Low	High	Pre-cache common Penasihat responses
Phase 4 (vote + approval) eats Sunday morning	Medium	Medium	Cut to manual admin button · preserve Phase 5 window
Demo machine fails on stage	Low	Critical	Backup video pre-recorded · narration self-contained
Pitch overrun (>4 minutes)	Medium	High	Strict timer rehearsals (2x Sunday morning) · recovery phrases per slide
Penasihat false positives (impractical items)	Medium	Medium	Curate seeded catalogue tightly · constrain Qwen prompt
Kampung trust score reads as punitive	Low	High	Frame collectively · high-trust unlocks better terms (not low-trust penalised)
Alibaba FC region down	Very low	Medium	Auto-failover to Anthropic Claude built into router
AWS ap-southeast-1 region down	Very low	Critical	Cross-cloud OSS backup mirror · DR drill documented
Stack reconciliation between team's pivot Drizzle→Prisma	Low (resolved)	—	Team standardised on Prisma per ijam2 branch

Open Questions Pre-Demo

1. **TNG PayLater sandbox** — does TNG actually have one? If not, our pitch frames DuitLater as a feature TNG could ship.
2. **MyKasih API access** — confirm with organisers if MyKasih Foundation is among FINHACK partners.
3. **NADI partnership** — verify whether MCMC/NADI is an event sponsor. If yes, request post-hackathon pilot intro.
4. **Pitch length** — 4 minutes assumed. Verify at organiser briefing Day 1.
5. **Demo room setup** — projector aspect ratio, audio output, backup laptop policy.
6. **Judging rubric weights** — verify if Financial Inclusion track judges weigh certain criteria higher.
7. **Post-hackathon IP terms** — confirm hackathon T&Cs.

These are answered/resolved at the organiser briefing on Day 1.

What We're Asking For (Technical)

Beyond the business pitch's 3 asks

TNG Engineering - Sandbox PayLater API access for the pool transaction flow (extended beyond demo-mode) - Webhook signing key for HMAC verification - Risk model documentation for individual allowance calibration

MCMC / NADI - API access to NADI centre directory + staff identities - Formal collaboration framework for kampung pilot - Co-authoring of NADI staff training material

MyKasih Foundation - Catalogue API (140,000+ items · 15 categories · 10,000+ merchant network) - Subsidy logic integration for MySARA-eligible household segmentation - Last-mile delivery partnership

Alibaba Cloud (Platinum sponsor) - DashScope API quota expansion for Qwen - Function Compute production tier (currently development) - Co-marketing for the multi-cloud reference architecture

AWS (Gold sponsor) - Activate \$5,000 sponsor credit for EC2 + S3 + CloudFront - IAM role assist for cross-cloud OSS access - AWS Activate program for post-hackathon scale path

Team R2-D2 • Five Hands

Member	Role	Archetype	Domain
Ijam (Zarul Izham)	Business Pitch Lead · Product Owner	<i>Narrative Spine</i>	Pitch · positioning · stakeholder framing
Mung (Khairul Anuar)	Backend Lead	<i>Foundation-Keeper</i>	Hono · Prisma · Postgres replication · TNG · multi-cloud orchestration
Akmal	Frontend Lead	<i>Surface-Weaver</i>	Next.js 15 · Tailwind v4 · interaction craft · NADI portal UI
Kairu	Product Manager · Phase Discipline	<i>Ladder-Keeper</i>	Phase plan · testable outcomes · scope guard · phase-gate verifier
MatNep	Classical Designer	<i>Orthodox Eye</i>	Brand · typography · grid · accessibility-as-craft · pitch deck design

Daily coordination via maji-core: - `/maji-onboard` — first-time intake with role card delivery - `/maji-phase` — cross-team BMAD phase status - `/maji-gate` — phase advancement requires testable outcome verification + independent verifier - `/maji-pair` — atomic logging of collaboration between members - `/maji-handoff` — end-of-session save with optional blocker logging

Akal protocol on every code change · **Jimat penuh** register default · personal memory committed to git for cross-team visibility on `git pull`.

DuitLater · Beyond the Hackathon

12-month vision

- Default group-purchase mechanism for B40 households on TNG eWallet
- 188 NADI centres serving as kampung-level entry points
- MyKasih merchant network as last-mile fulfilment
- Kampung-level trust scores recognised by TNG as input for individual PayLater limit calibration

Phase-aware roadmap

- Q3 2026 — pilot at 1 NADI Felda Gedangsa
- Q4 2026 — expand to 5 NADI centres in Selangor

- Q1 2027 — formal partnership review with MCMC + MyKasih + TNG
- Q2 2027 — national soft launch (188 NADI centres)

Sustainability

- No new welfare programme invented
- No new credit instrument
- Composes existing Malaysian institutions (TNG · NADI · MyKasih)
- Funded under permanent mandates of all three partners

“Sendiri tak mampu, ramai-ramai boleh.”

The platform begins as a hackathon prototype. It scales as a Malaysian institutional combination already aligned by mandate.

Repository · Documentation Trail

Code repos - Team: github.com/Zen0space/R2-D2-Finhack - Personal mirror: github.com/ljam18/duitlater (ljam's branch) - Branches: `main` · `dev` · `ijam` · `ijam2` · `akmal`

Documentation in repo (*judges may verify all claims*)

- `README.md` — repo entry + 3-layer AI summary
- `PRD.md` — 617-line Product Requirements Document
- `ARCHITECTURE.md` — system architecture · 8 mermaid diagrams · ER · multi-cloud topology · scale path · security posture
- `WORLD.md` — manifesto · cultural anchor · institutional package narrative
- `BRAND.md` — visual identity · palette · typography · iconography

- `TEAM.md` — roster · phase ownership · norms · decision log
- `DEVELOPMENT-PLAN.md` — 7-phase build · cut-line strategy · phase status table
- `docs/pitch/business-pitch-deck.{md,pdf}` — this 8-slide pitch
- `docs/pitch/technical-pitch-deck.{md,pdf}` — this technical deck
- `docs/tech/multi-cloud-setup.{md,pdf}` — 1,400-line setup guide for Mung
- `docs/ai-methodology.md` — 3-layer AI integration explained
- `infra/RELEASE.md` — single-EC2 baseline runbook
- `alibaba-function-compute/` — deployable Qwen wrapper

Team coordinator - `maji-core/` — protocols · heroes · commands · memory · schema

Documentation quality at hackathon = exceptional. Documentation as evidence for the **Presentation & Teamwork** judging criterion.

Thank You

DuitLater

Sendiri tak mampu, ramai-ramai boleh.

Pitch lead: Ijam · zarulijam@gmail.com **Repo:** github.com/Zen0space/R2-D2-Finhack **Track:** Financial Inclusion · TNG FINHACK 2026

Multi-cloud orchestration. HA failover. BM-native AI. Three Malaysian institutions composed. One product.

Designed criterion-first. Shipped institutionally honest.

Questions welcome. Documentation traceable.